



Reflexión final del equipo MyDoomTeam (ArenaSync)

Aprendizajes obtenidos durante el desarrollo

El desarrollo de la plataforma ArenaSync nos demostró que la construcción de software no se limita únicamente a programar, sino que exige diseñar, validar, comunicar y mejorar de forma continua. Entre nuestros principales aprendizajes destacamos:

- **Gestión de proyectos:** Comprendimos la vital importancia de planificar el trabajo mediante iteraciones cortas, definiendo prioridades claras y entregables específicos. En la industria tecnológica actual, las metodologías ágiles ratifican que las entregas pequeñas reducen el riesgo de fallos masivos y mejoran el ciclo de retroalimentación.
- **Comunicación y trabajo en equipo:** Identificamos que es fundamental comunicar cualquier cambio en los contratos de datos de manera temprana para evitar la desalineación entre los desarrolladores de Frontend y Backend.
- **Disciplina y documentación:** Aprendimos que documentar las decisiones de diseño y arquitectura es esencial para evitar el retrabajo y mantener un criterio unificado en todo el equipo. Entendimos que no existe una verdadera velocidad de desarrollo si la base técnica es defectuosa (está en rojo); mantener la calidad continua realmente ahorra tiempo a largo plazo.
- **Habilidades técnicas:** Consolidamos nuestra experiencia en el diseño de arquitecturas por capas (Monolito), la implementación de pruebas automatizadas, la trazabilidad del código y la ejecución de refactorizaciones estructurales profundas.

Evaluación del temario y sugerencias para futuras ediciones

Estamos muy satisfechos con los temas abordados y consideramos que mantener el enfoque netamente práctico es el mayor valor que ofrece el curso actualmente. Sin embargo, para preparar mejor a los estudiantes para los entornos de producción reales y el mundo profesional, consideramos importante añadir y profundizar en los siguientes temas para futuras ediciones:

- **Preparación para el mercado laboral:** Queremos destacar de manera especial que la última sección, en la cual el profesor habló de temas laborales, el estado del mercado y cómo construir un CV, fue de muchísima ayuda. Sugerimos fuertemente mantener este espacio y, en lo posible, profundizar más en estas temáticas en otras ocasiones para fortalecer el perfil profesional de los estudiantes.
- **Observabilidad y Métricas:** Sugerimos incluir un bloque formal sobre la definición de Acuerdos de Nivel de Servicio (SLA), implementación de logs estructurados y sistemas de tracing. Complementando esto, la observabilidad es indispensable hoy en día para diagnosticar cuellos de botella y garantizar la alta disponibilidad de los sistemas en la nube.
- **Pruebas de Contrato (Frontend-Backend):** Recomendamos reforzar la fase de integración enseñando a implementar pruebas de contrato automatizadas entre las capas. Esto simula el trabajo real de microservicios y equipos distribuidos, mitigando el "infierno de integración".



- **Arquitectura Evolutiva y Deuda Técnica:** Sería sumamente valioso incorporar sesiones guiadas sobre cómo identificar, medir y pagar la deuda técnica, así como principios de arquitectura evolutiva.

Problemas, limitaciones o restricciones en el proceso de aprendizaje

Nuestros errores más costosos no fueron estrictamente de código, sino que derivaron de fallos en la coordinación, la disciplina y los procesos internos del equipo. Estas limitaciones representaron fricciones importantes:

- **Inestabilidad de las pruebas automatizadas:** Durante las etapas intermedias del ciclo de desarrollo, mantuvimos una suite de pruebas inestable, lo que nos obligó a detener la creación de nuevas características para corregir y priorizar un estado verde continuo mediante herramientas como pytest y Ruff.
- **Desalineación en contratos de API:** Tuvimos serias dificultades técnicas para alinear las peticiones HTTP y las respuestas entre el Backend y el Frontend sin romper los flujos que ya estaban contruidos. Esto ocurrió principalmente en endpoints críticos y nos forzó a estandarizar los esquemas de datos.
- **Gestión del estado y reglas de negocio:** Controlar las transiciones complejas del estado de los torneos fue un obstáculo técnico considerable, debido a la necesidad de aplicar reglas de negocio muy estrictas de manera sincrónica.
- **Mantenibilidad del código (Código espagueti):** Generamos métodos demasiado largos que dificultaron la lectura y el proceso de aprendizaje general. Como restricción autoimpuesta para mejorar, tuvimos que invertir bastante tiempo en descomponer estos métodos utilizando helpers y aplicando el principio de responsabilidad única, cuidando de no alterar el comportamiento ni la cobertura de las pruebas.